2D Cricket Web Application Report

Student Information

- **Name:** Minahil Kashif
- **Roll Number:** 23i-0554
- **Section:** C

1. Introduction

This project is a 2D Cricket Batting Web Application developed using **React, JavaScript, HTML, and CSS**. The game simulates a simplified cricket batting experience where the player selects a batting style and plays shots based on a probability-driven power bar system.

The objective of this project is to demonstrate concepts of:

- UI Design
- Event Handling
- Animations
- Structured Programming
- Probability-based Game Logic

2. Game Logic and Working Mechanism

The game is designed as a **single-player batting simulation**.

Core Flow:

1. The user selects a batting style:
    - Aggressive
    - Defensive
2. Each ball follows phases:
    - **Ready** → waiting for user
    - **Bowling** → ball animation
    - **Batting** → user clicks to play shot
    - **Result** → outcome displayed
3. When the user clicks “Play Shot”:
    - The current slider position (0–1) is captured.

    - This value determines the outcome based on predefined probability segments.
4. The game updates:
    - Runs
    - Wickets
    - Balls played
5. The game ends when:
    - 12 balls are completed (2 overs), OR
    - 2 wickets are lost
6. A restart option resets the game state.

3. Mapping of Probabilities to the Power Bar

Each batting style has predefined probabilities for outcomes:

- Wicket
- 0, 1, 2, 3, 4, 6 runs

Example (Aggressive Style):

- Wicket: High probability
- Boundaries (4, 6): Higher probability
- Lower safe outcomes

Implementation:

1. Probabilities are stored in objects.
2. A function converts them into **segments**:
    - Each segment has:
        - Start position
        - End position
        - Outcome
3. The power bar visually represents these segments:
    - Width of each segment = probability × 100%
    - Example:
        - 0.40 → 40% of bar
4. A slider continuously moves across the bar.
5. When the user clicks:
    - The slider position is checked
    - The segment containing that position determines the outcome

Important:

- No randomness is used for deciding results
- Outcome is purely based on slider position

4. Implementation of Animations

1. Power Bar Animation

- Implemented using `requestAnimationFrame`
- Slider moves continuously from left to right
- Resets after reaching the end

2. Bowling Animation

- Triggered when user clicks “Play Shot” in ready phase
- Ball visually travels toward the batsman
- Controlled using game phase state and timing

3. Batting Animation

- Triggered when user plays the shot
- Visual feedback includes:
    - Outcome overlay (runs or wicket)
    - Flash effects:
        - Red glow for wicket
        - Yellow glow for boundaries

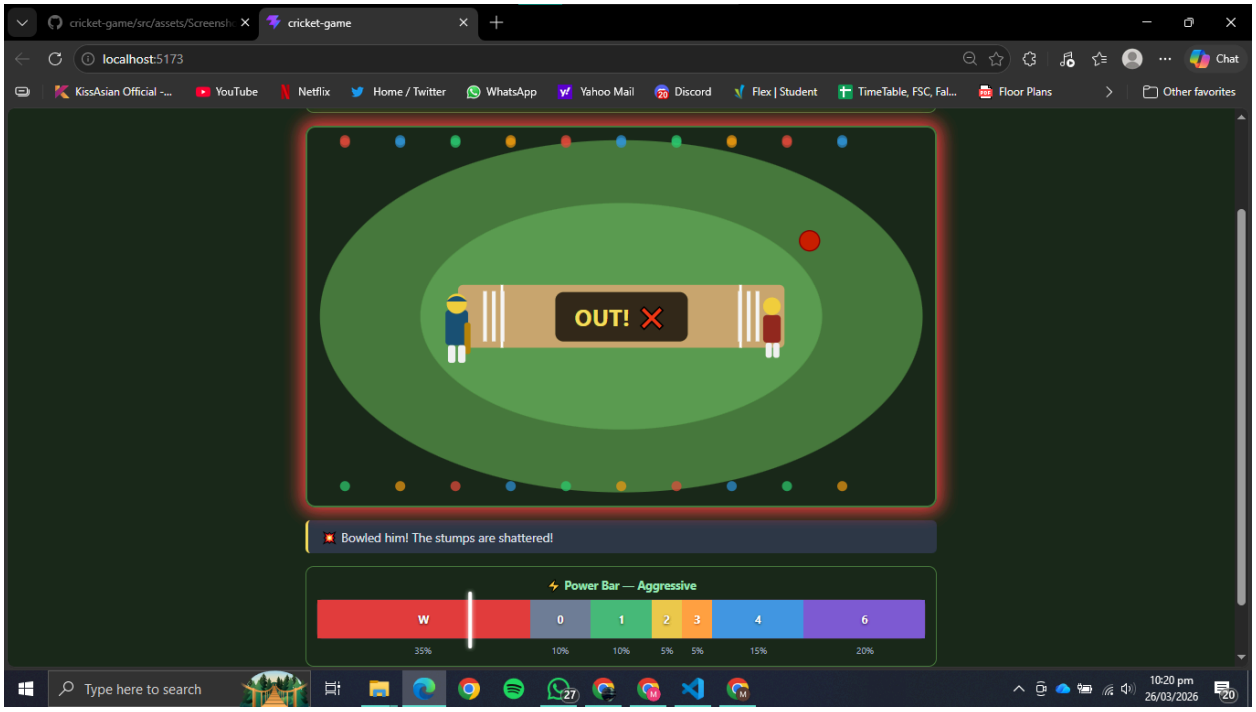
4. UI Feedback Animations

- Pop-in animation for result display
- Pulse animation for “Play Shot” button
- Smooth transitions for slider movement

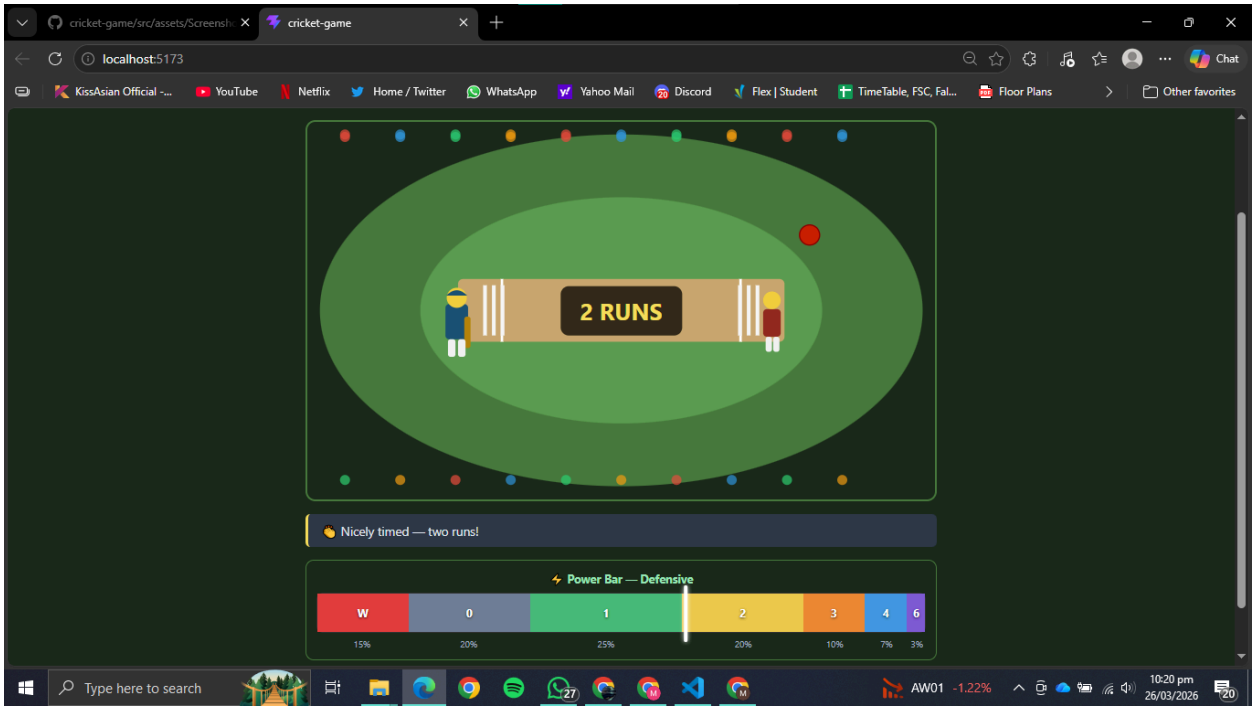
5. Screenshots

(Insert full-screen screenshots here with sticky note visible)

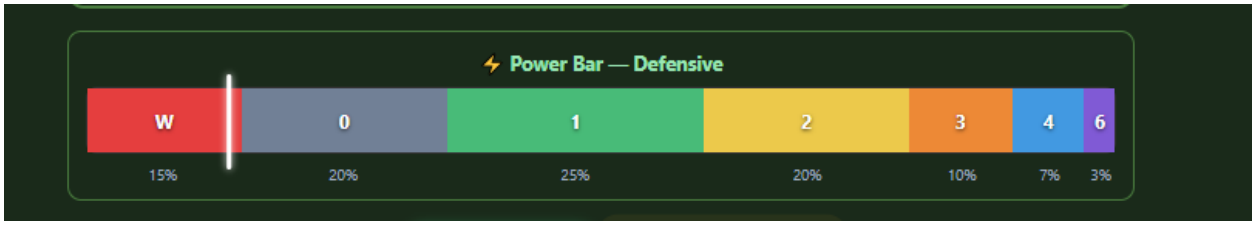
1. Aggressive Batting in Action



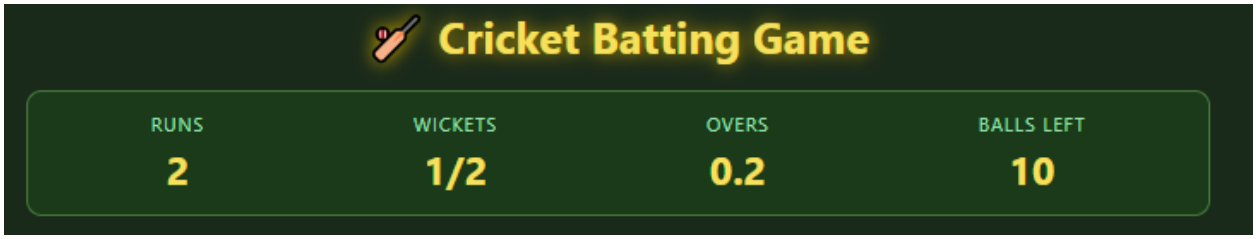
2. Defensive Batting in Action



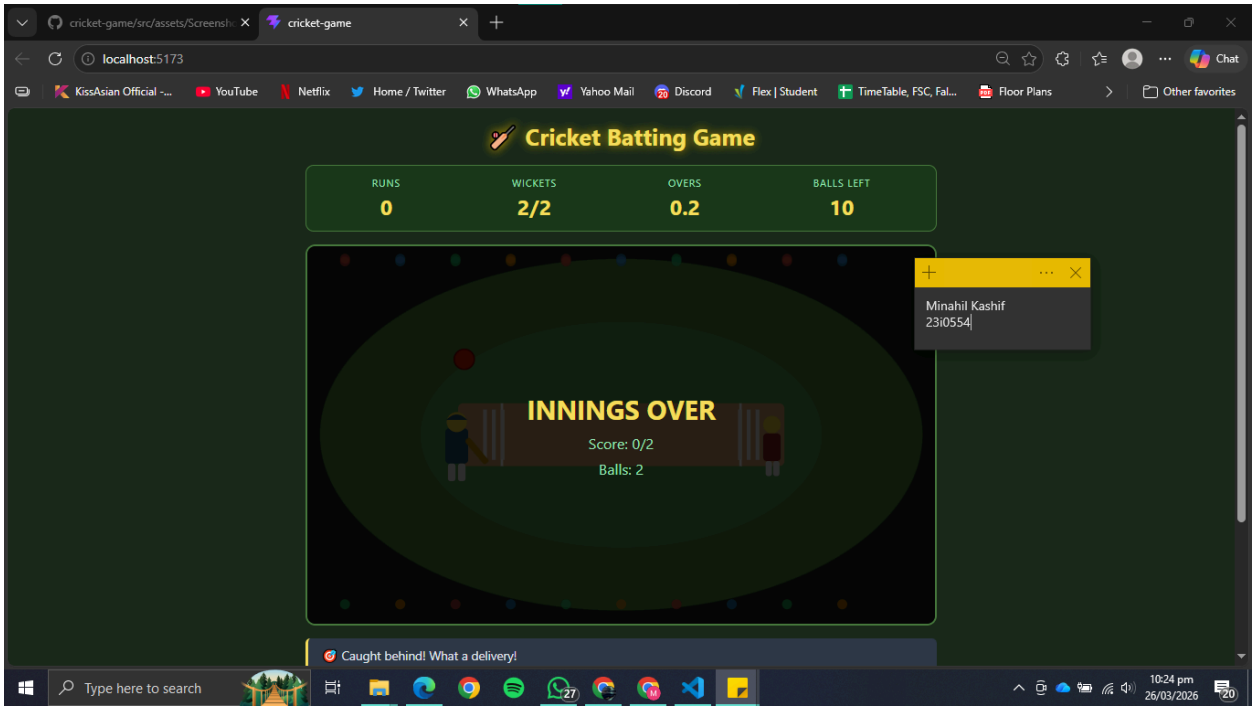
3. Power Bar During Gameplay



4. Scoreboard Updating



5. Game Over Screen



6. Conclusion

This project successfully demonstrates the integration of **probability-based logic with interactive UI design**. The use of a power bar instead of randomness ensures deterministic gameplay, while animations and structured state management enhance the user experience.

The application fulfills all functional requirements and includes additional features such as dynamic commentary and visual feedback.